

Programación para Física y Astronomía

Departamento de Física.

Coordinadora: C Loyola

Profesores C Femenías / D Basantes

Primer Semestre 2025

Universidad Andrés Bello

Departamento de Física y Astronomía



Introducción y Conexión

Ejercicios Prácticos

Soluciones de Referencia

Consolidación y Retroalimentación

Conclusiones

Introducción y Conexión

Conceptos fundamentales que repasamos

- Ajuste lineal con `np.polyfit`
- Ajuste flexible con `scipy.optimize.curve_fit`
- Métricas de calidad: residuos, R^2 , χ^2_{red}
- Ponderación por incertidumbres variables
- Extracción de incertidumbres desde matriz de covarianza
- Análisis de residuos para diagnóstico de modelos

Objetivo de hoy

Consolidar estos conceptos con ejercicios prácticos integradores y más complejos.

- **Consolidar** el uso de `curve_fit` en problemas físicos variados
- **Aplicar** ponderación y cálculo de χ^2_{red} sistemáticamente
- **Desarrollar** habilidades de diagnóstico con análisis de residuos
- **Integrar** múltiples conceptos en aplicaciones prácticas complejas

Ejercicios Prácticos

- **Enfoque 100% práctico:** Resolver 6 problemas de complejidad creciente
- **Integración de conceptos:** Combinar ajuste + métricas + diagnóstico
- **Aplicación contextualizada:** Física, astronomía, experimentos reales

Estructura pedagógica

- Ejercicios 1-2: Ajustes básicos con `curve_fit` y R^2
- Ejercicios 3-4: Incorporar ponderación y χ^2_{red}
- Ejercicios 5-6: Modelos complejos y selección de modelos



Enunciado

- Un resorte sigue $F = kx$. Datos:
 - $x = [0.02, 0.05, 0.08, 0.10, 0.12, 0.15, 0.18, 0.20]$ m
 - $k_{\text{real}} = 250$ N/m, $\sigma_F = 2.0$ N
- Tareas:
 1. Generar datos F con ruido (seed=42)
 2. Ajustar con `curve_fit` para estimar k
 3. Calcular R^2
 4. Graficar datos (scatter) y ajuste (línea)
 5. Reportar k_{est} y R^2

Conceptos: Modelo proporcional directo, ajuste de 1 parámetro.

Física relevante: Elasticidad, Ley de Hooke.



Enunciado

- Posición: $y(t) = y_0 - \frac{1}{2}gt^2$. Datos:
 - $t = [0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0]$ s
 - $y_0 = 50.0$ m, $g = 9.8$ m/s², $\sigma_y = 1.0$ m
- Tareas:
 1. Generar datos y con ruido (seed=15)
 2. Ajustar modelo `caida(t, y0, g)`
 3. Calcular R^2
 4. Graficar datos y curva ajustada
 5. Comparar g_{est} con valor real

Conceptos: Ajuste cuadrático, 2 parámetros.

Conexión: Más complejo que Ej. 1 (múltiples parámetros).



Enunciado

- Stefan-Boltzmann: $P = \sigma AT^4$, ajustar $P = c \cdot T^4$. Datos:
 - $T = [300, 400, 500, 600, 700, 800, 900, 1000]$ K
 - $c_{\text{real}} = 5.67 \times 10^{-10}$ W/K⁴
 - $\sigma_P = (0.5 + 0.001 \cdot T) \times 10^{-6}$ W
- Tareas:
 1. Generar datos P con ruido (seed=20)
 2. Ajustar CON ponderación (**sigma=sigma_P**)
 3. Extraer c y σ_c desde **pcov**
 4. Calcular χ^2_{red}
 5. Graficar con **errorbar**
 6. Reportar $c \pm \sigma_c$

Conceptos: Ley de potencia, ponderación, matriz de covarianza.



Enunciado

- Voltaje: $V(t) = V_0 e^{-t/\tau}$. Datos:
 - $t = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$ s
 - $V_0 = 12.0$ V, $\tau = 3.0$ s
 - $\sigma_V = 0.3 + 0.02 \cdot t$ V
- Tareas:
 1. Generar datos (seed=25)
 2. Ajustar SIN y CON ponderación
 3. Comparar τ en ambos casos
 4. Calcular χ^2_{red} ponderado
 5. Crear 2 subplots: datos+ajuste y residuos

Conceptos: Decaimiento exponencial, comparación de métodos.

Conexión: Similar a Ej. 3 pero con análisis comparativo.



Enunciado

- Velocidad: $v(r) = \sqrt{GM \left(\frac{2}{r} - \frac{1}{a} \right)}$. Datos:
 - $r = [0.8, 0.9, 1.0, 1.1, 1.2, 1.3, 1.4, 1.5, 1.6, 1.7]$ AU
 - $GM = 1.0$, $a = 1.2$ AU, $\sigma_v = 0.05$ AU/año
- Tareas:
 1. Generar datos (seed=30)
 2. Ajustar con $\mathbf{p0} = [1.5, 1.5]$
 3. Reportar GM y a con incertidumbres
 4. Calcular χ^2_{red}
 5. Graficar con intervalo fino de r

Conceptos: Modelo no lineal complejo, importancia de $\mathbf{p0}$.

Física: Mecánica orbital, Kepler generalizada.



Enunciado

- Modelo completo: $x(t) = Ae^{-\gamma t} \cos(\omega t + \phi)$
- Modelo simple: $x(t) = Ae^{-\gamma t}$
- Datos: $t = \text{np.linspace}(0, 10, 50)$, $A = 5$, $\gamma = 0.3$, $\omega = 2$, $\phi = 0.5$
- Tareas:
 1. Generar datos con modelo completo (seed=35)
 2. Ajustar ambos modelos
 3. Calcular AIC: $\text{AIC} = 2k + n \ln(\chi^2/n)$
 4. Comparar ΔAIC
 5. Seleccionar mejor modelo
 6. Graficar ambos ajustes

Conceptos: Selección de modelos, trade-off complejidad-ajuste.

Aplicación: Evitar sobre-ajuste con criterio objetivo.

Soluciones de Referencia



Ley de Hooke

```
1 import numpy as np
2 from scipy.optimize import curve_fit
3 import matplotlib.pyplot as plt
4
5 # Datos
6 x = np.array([0.02, 0.05, 0.08,
7               0.10, 0.12, 0.15,
8               0.18, 0.20])
9 k_real = 250.0
10 sigma_F = 2.0
11
12 # Generar F con ruido
13 np.random.seed(42)
14 F_true = k_real * x
15 F_obs = F_true + np.random.normal(
16     0, sigma_F, size=x.size)
```

```
15 # Modelo
16 def hooke(x, k):
17     return k * x
18
19 # Ajustar
20 popt, _ = curve_fit(hooke, x, F_obs)
21 k_est = popt[0]
22 F_fit = hooke(x, k_est)
23
24 # R²
25 R2 = 1 - np.sum((F_obs - F_fit)**2) / \
26     np.sum((F_obs - np.mean(F_obs))**2)
27
28 print(f"k = {k_est:.2f} N/m")
29 print(f"R² = {R2:.4f}")
```



Caída Libre

```
1 import numpy as np
2 from scipy.optimize import curve_fit
3
4 # Datos
5 t = np.array([0.0, 0.5, 1.0, 1.5,
6              2.0, 2.5, 3.0])
7 y0_true, g_true = 50.0, 9.8
8 sigma_y = 1.0
9
10 # Generar y
11 np.random.seed(15)
12 y_true = y0_true - 0.5 * g_true * t**2
13 y_obs = y_true + np.random.normal(
14     0, sigma_y, size=t.size)
15
16
17 # Modelo
18 def caida(t, y0, g):
19     return y0 - 0.5 * g * t**2
20
21 # Ajustar
22 popt, _ = curve_fit(caida, t, y_obs)
23 y0_est, g_est = popt
24 y_fit = caida(t, y0_est, g_est)
25
26 # R^2
27 R2 = 1 - np.sum((y_obs - y_fit)**2) / \
28     np.sum((y_obs - np.mean(y_obs))**2)
29
30 print(f"g = {g_est:.3f} m/s^2")
31 print(f"R^2 = {R2:.4f}")
```

Discusión: g_{est} debe estar cerca de 9.8 m/s^2 .

Soluciones de Referencia $\ni$ Solución 3:

Radiación de Cuerpo Negro



```
1 import numpy as np
2 from scipy.optimize import curve_fit
3
4 # Datos
5 T = np.array([300, 400, 500, 600,
6               700, 800, 900, 1000])
7 c_real = 5.67e-10
8 sigma_P = (0.5 + 0.001 * T) * 1e-6
9
10 # Generar P
11 np.random.seed(20)
12 P_true = c_real * T**4
13 P_obs = P_true + np.random.normal(
14     0, 1, size=T.size) * sigma_P
15
16 # Modelo
17 def stefan(T, c):
18     return c * T**4
```

```
16 # Ajustar con ponderación
17 popt, pcov = curve_fit(
18     stefan, T, P_obs,
19     sigma=sigma_P,
20     absolute_sigma=True)
21 c_est = popt[0]
22 sigma_c = np.sqrt(pcov[0, 0])
23
24 # Chi² reducido
25 P_fit = stefan(T, c_est)
26 chi2 = np.sum(((P_obs - P_fit) /
27               sigma_P)**2)
28 chi2_red = chi2 / (T.size - 1)
29
30 print(f"c = {c_est:.2e} ± "
31       f"{sigma_c:.2e} W/K⁴")
32 print(f"χ²_red = {chi2_red:.2f}")
```

Soluciones de Referencia $\ni$ Solución 4:

Circuito RC (Parte 1)



```
1 import numpy as np
2 from scipy.optimize import curve_fit
3
4 # Datos
5 t = np.array(range(11), dtype=float)
6 V0_true, tau_true = 12.0, 3.0
7 sigma_V = 0.3 + 0.02 * t
8
9 # Generar V
10 np.random.seed(25)
11 V_true = V0_true * np.exp(-t/tau_true)
12 V_obs = V_true + np.random.normal(
13     0, 1, size=t.size) * sigma_V
14
15 # Modelo
16 def rc_decay(t, V0, tau):
17     return V0 * np.exp(-t/tau)
```

```
16 # Ajuste SIN ponderación
17 popt_sin, _ = curve_fit(
18     rc_decay, t, V_obs)
19 V0_sin, tau_sin = popt_sin
20
21 # Ajuste CON ponderación
22 popt_con, pcov_con = curve_fit(
23     rc_decay, t, V_obs,
24     sigma=sigma_V,
25     absolute_sigma=True)
26 V0_con, tau_con = popt_con
27 sigma_tau = np.sqrt(pcov_con[1, 1])
28
29 print("Sin ponderación:")
30 print(f"   $\tau = \{\text{tau\_sin:.3f}\} \text{ s}")$ 
31 print("Con ponderación:")
32 print(f"   $\tau = \{\text{tau\_con:.3f}\} \pm "$ 
33     f" $\{\text{sigma\_tau:.3f}\} \text{ s}")$ 
```

Soluciones de Referencia $\ni$ Solución 4:

Circuito RC (Parte 2: Gráficos)



```
1 import matplotlib.pyplot as plt
2
3 # Chi² reducido
4 V_fit_con = rc_decay(t, V0_con, tau_con)
5 chi2 = np.sum(((V_obs - V_fit_con) / sigma_V)**2)
6 chi2_red = chi2 / (t.size - 2)
7 print(f"χ²_red = {chi2_red:.2f}")
8
9 # Gráficos
10 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
11
12 # Panel 1: Datos y ajuste
13 axes[0].errorbar(t, V_obs, yerr=sigma_V, fmt='o', capsize=3, label='Datos')
14 axes[0].plot(t, V_fit_con, 'r-', linewidth=2, label=f'τ={tau_con:.2f}±{sigma_tau:.2f} s')
15 axes[0].set_xlabel('Tiempo (s)')
16 axes[0].set_ylabel('Voltaje (V)')
17 axes[0].legend()
18 axes[0].grid(True, alpha=0.3)
19
20 # Panel 2: Residuos
21 res = V_obs - V_fit_con
22 axes[1].scatter(t, res, s=40)
23 axes[1].axhline(0, color='r', linestyle='--', linewidth=2)
24 axes[1].set_xlabel('Tiempo (s)')
25 axes[1].set_ylabel('Residuos (V)')
26 axes[1].grid(True, alpha=0.3)
27
28 plt.tight_layout()
29 plt.show()
```

Discusión: Ajuste ponderado da mejor estimación de τ .

Soluciones de Referencia $\ni$ Solución 5:

Velocidad Orbital



```
1 import numpy as np
2 from scipy.optimize import curve_fit
3
4 # Datos
5 r = np.array([0.8, 0.9, 1.0, 1.1,
6              1.2, 1.3, 1.4, 1.5,
7              1.6, 1.7])
8 GM_true, a_true = 1.0, 1.2
9 sigma_v = 0.05
10
11 # Generar v
12 np.random.seed(30)
13 v_true = np.sqrt(GM_true * (2/r - 1/a_true))
14 v_obs = v_true + np.random.normal(
15     0, sigma_v, size=r.size)
```

```
14 # Modelo
15 def orbital_vel(r, GM, a):
16     return np.sqrt(GM * (2/r - 1/a))
17
18 # Ajustar con p0
19 popt, pcov = curve_fit(
20     orbital_vel, r, v_obs,
21     p0=[1.5, 1.5],
22     sigma=np.full(r.size, sigma_v),
23     absolute_sigma=True)
24 GM_est, a_est = popt
25 perr = np.sqrt(np.diag(pcov))
26
27 print(f"GM = {GM_est:.3f}  $\pm$  "
28       f"{perr[0]:.3f}")
29 print(f"a = {a_est:.3f}  $\pm$  "
30       f"{perr[1]:.3f} AU")
```

Discusión: Valores iniciales apropiados son cruciales para convergencia.

Soluciones de Referencia $\ni$ Solución 6:

Selección de Modelo (Parte 1)



```
1 import numpy as np
2 from scipy.optimize import curve_fit
3
4 # Datos
5 t = np.linspace(0, 10, 50)
6 A, gamma, omega, phi = 5.0, 0.3, 2.0, 0.5
7 sigma_x = 0.3
8
9 # Generar datos
10 np.random.seed(35)
11 x_true = A * np.exp(-gamma * t) * \
12     np.cos(omega * t + phi)
13 x_obs = x_true + np.random.normal(
14     0, sigma_x, size=t.size)
```

```
13 # Modelo 1: completo
14 def modelo1(t, A, gamma, omega, phi):
15     return A * np.exp(-gamma * t) * \
16         np.cos(omega * t + phi)
17
18 # Modelo 2: simple
19 def modelo2(t, A, gamma):
20     return A * np.exp(-gamma * t)
21
22 # Ajustar ambos
23 popt1, _ = curve_fit(
24     modelo1, t, x_obs,
25     p0=[5, 0.3, 2, 0.5])
26 popt2, _ = curve_fit(
27     modelo2, t, x_obs, p0=[5, 0.3])
```

Soluciones de Referencia $\ni$ Solución 6: Selección de Modelo (Parte 2)



```
1 # Calcular chi² para cada modelo
2 x_fit1 = modelo1(t, *popt1)
3 x_fit2 = modelo2(t, *popt2)
4
5 chi2_1 = np.sum(((x_obs - x_fit1) / sigma_x)**2)
6 chi2_2 = np.sum(((x_obs - x_fit2) / sigma_x)**2)
7
8 # Calcular AIC
9 n = t.size
10 k1, k2 = 4, 2 # número de parámetros
11 AIC1 = 2*k1 + n*np.log(chi2_1/n)
12 AIC2 = 2*k2 + n*np.log(chi2_2/n)
13
14 delta_AIC = AIC2 - AIC1
15
16 print(f"Modelo 1 (completo): AIC = {AIC1:.2f},  $\chi^2$  = {chi2_1:.2f}")
17 print(f"Modelo 2 (simple): AIC = {AIC2:.2f},  $\chi^2$  = {chi2_2:.2f}")
18 print(f" $\Delta$ AIC = {delta_AIC:.2f}")
19
20 if delta_AIC > 10:
21     print("→ Modelo 1 es significativamente mejor")
22 elif delta_AIC < -10:
23     print("→ Modelo 2 es significativamente mejor")
24 else:
25     print("→ Evidencia no concluyente")
```

Discusión: AIC penaliza complejidad, favorece modelo que equilibra ajuste y parsimonia.

Consolidación y Retroalimentación

- ¿Qué ejercicio resultó más desafiante?
- ¿Encontraron diferencias significativas entre ajustes con/sin ponderación?
- ¿El análisis de residuos reveló problemas en algún modelo?
- ¿Dificultades con la interpretación de χ^2_{red} o matriz de covarianza?
- ¿La selección de modelo con AIC fue clara?

Objetivo

Aprender de las experiencias colectivas y consolidar diagnóstico de modelos.

Errores frecuentes

- No usar **absolute_sigma=True**: incertidumbres mal escaladas
- Olvidar valores iniciales (**p0**): modelos complejos no convergen
- Ignorar análisis de residuos: perder patrones sistemáticos
- Interpretar mal $\chi^2_{\text{red}} \gg 1$: puede indicar subestimación de σ
- Sobre-ajustar sin criterio objetivo: AIC ayuda a evitarlo

Consejo

Siempre: ajuste $\rightarrow$ métricas $\rightarrow$ residuos $\rightarrow$ interpretación física $\rightarrow$ decisión informada.

Conclusiones

- **Consolidamos:** aplicación sistemática de `curve_fit` con diagnóstico completo
 - Ajustes lineales y no lineales
 - Ponderación rigurosa
 - Extracción de incertidumbres
- **Aplicamos:** métricas múltiples para evaluación objetiva
 - R^2 para bondad general
 - χ^2_{red} para consistencia con errores
 - AIC para selección de modelos
- **Desarrollamos:** habilidades críticas de diagnóstico con análisis de residuos

Competencias adquiridas

Ahora pueden:

- Ajustar cualquier modelo físico con `curve_fit`
- Manejar incertidumbres variables correctamente
- Diagnosticar calidad de ajuste con múltiples métricas
- Extraer e interpretar incertidumbres de parámetros
- Seleccionar entre modelos competidores objetivamente
- Analizar residuos para detectar problemas sistemáticos

Para la evaluación

Practiquen el flujo completo: datos $\rightarrow$ modelo $\rightarrow$ ajuste $\rightarrow$ métricas $\rightarrow$ diagnóstico $\rightarrow$ conclusión.

- Documentación oficial:
 - `scipy.optimize.curve_fit`
 - `np.polyfit`
- Teoría estadística:
 - Bevington & Robinson: "Data Reduction and Error Analysis"
 - Taylor: "An Introduction to Error Analysis"
- Práctica:
 - Rehacer ejercicios con otros valores de parámetros
 - Experimentar con diferentes niveles de ruido
 - Aplicar a datos reales de laboratorio